

BUILDING REST APIs WITH DJANGO

FIVE-DAY LEARNING AGENDA

TASK 02

TEACH.
BUILD.
DELIVER.

ORGANIZATION	TRACK	PROGRAM DATES	DUE DATE
Tuwaiq Club at KFUPM	Modern Software & Cloud	October 11-15, 2026	September 30, 2026

01 PROGRAM OUTCOME

BY THE END OF DAY 5

Participants can design, build, test, document, and deploy a database-backed REST API using Django REST Framework, including token authentication, validated CRUD operations, and per-user permissions.

02 FIVE-DAY LEARNING JOURNEY

DAY	FOCUS	PROJECT INCREMENT	DAILY PROOF
1	Foundations and setup	Repository, Django/DRF project, Task model, migrations, admin data	Server runs; Task records persist
2	Core API	Serializers, routes, and complete local CRUD endpoints	CRUD checklist passes
3	Realistic behavior	Token auth, ownership, data isolation, permissions, validation	Two-user security check passes
4	Quality and delivery	Automated tests, README, environment and deployment configuration	Tests pass; fresh setup succeeds
5	Publish and demonstrate	Production deployment, smoke tests, final fixes, demonstration	Live URL and final evidence pass

03 DELIVERY MODEL

PER-DAY LEARNING BALANCE

Keep instructor explanation and demonstration focused at approximately **30%** of the session. Reserve at least **70%** for guided exercises, project construction, troubleshooting, and practical verification. The daily pages intentionally show only these two categories.

FACILITATION RULES

- Use short explanations immediately followed by visible implementation.
- Keep one project repository growing across all five days.
- End each day with a practical exit check, not a knowledge-only quiz.
- Provide a known-good checkpoint branch so blocked participants can recover.
- Do not begin optional extensions until the day's required gate passes.

04 PROJECT AND SCOPE CONTROLS

PERSONAL TASK MANAGEMENT REST API

Each authenticated user manages only their own tasks. The final API includes a persistent Task model, complete CRUD operations, token authentication, per-user permissions, input validation, at least eight automated tests, a reproducible GitHub repository, a clear README, and a public HTTPS deployment. A frontend is not required; optional features never block completion.

DAY 1

FOUNDATIONS, SETUP, AND PROJECT START

Sunday, Oct 11

DAILY LEARNING OUTCOME

By the end of the session, participants can explain the basic HTTP request-response flow, run a reproducible Django REST Framework environment, and persist Task records through a correctly migrated data model.

TEACHING AND PRACTICAL PLAN

TEACHING CONTENT (~30%)

WHAT THE INSTRUCTOR EXPLAINS

- Begin with the finished project: a user sends an HTTP request, Django receives it, DRF validates it, the ORM reads or writes the database, and the API returns JSON plus a status code.
- Use the Task resource to connect HTTP verbs to intent: GET reads, POST creates, PUT/PATCH changes, and DELETE removes. Emphasize that the URL identifies the resource while the verb describes the action.
- Clarify the Django structure: the project stores global configuration; the tasks app owns task behavior; DRF adds API-specific tools on top of Django.
- Explain that a model is the Python definition of stored data, the ORM lets Python work with rows, and migrations safely translate model changes into database changes.
- Explain why reproducibility matters: dependencies must be recorded, generated environments must not be committed, and secrets must stay outside Git.

WHAT THE INSTRUCTOR DEMONSTRATES

- Create a virtual environment and dependency file
- Start the Django project and tasks app; enable DRF
- Define Task fields and status choices; connect owner to Django User
- Run migrations, register Task in admin, and create sample records
- Commit a clean Day 1 checkpoint

PRACTICAL WORK (70%+)

GUIDED START

Participants follow a setup checklist in pairs. After each checkpoint, they compare folder structure, settings, migration output, and admin behavior with the instructor reference before moving forward.

PARTICIPANTS THEN BUILD AND VERIFY

- Create and activate a virtual environment; install the approved dependencies; record them in the dependency file.
- Initialize and push a GitHub repository with an appropriate .gitignore.
- Create the Django project and tasks app; enable DRF and verify the development server.
- Implement every required Task field and status choice, create the owner relationship, and generate/apply migrations.
- Register Task in admin; create two users and realistic sample tasks; restart the server and confirm data persists.
- Compare the repository against the Day 1 checklist and push the completed checkpoint.

PROJECT FEATURE ADDED

Repository and reproducible environment; Django/DRF project; tasks app; Task model with id, title, description, status, due_date, owner, created_at, and updated_at; migrations; admin registration; sample data.

EXPECTED RESULT

A clean repository launches locally, the database schema is current, and at least two users and several Task records can be created and viewed through Django admin.

COMPLETION CRITERIA

- Fresh terminal session activates the environment and starts the server
- `python manage.py migrate` reports no pending migrations
- Task fields and status choices match the approved project scope
- Admin can create, update, and persist Task records with owners
- Repository is pushed with no secrets or environment folder committed

NEXT-DAY CONNECTION

Day 2 uses this model as the API resource. Participants will convert Task instances to JSON with a serializer and expose them through routed CRUD endpoints.

DAY 2

CORE API AND COMPLETE CRUD

Monday, Oct 12

DAILY LEARNING OUTCOME

By the end of the session, participants can convert Task objects to validated JSON and implement routed create, list, retrieve, update, partial-update, and delete operations with appropriate HTTP responses.

TEACHING AND PRACTICAL PLAN

TEACHING CONTENT (~30%)

WHAT THE INSTRUCTOR EXPLAINS

- Explain the serializer as both translator and gatekeeper: it turns model instances into JSON, converts incoming JSON into Python data, and rejects invalid input before anything is saved.
- Map each required operation to its route, verb, success status, and failure behavior so participants see the API as a consistent contract rather than unrelated functions.
- Explain how a viewset connects requests to queryset operations and how a router generates predictable list and detail URLs.
- Walk through one POST request from client JSON to serializer validation, model save, database row, serialized response, and 201 status.
- Contrast PUT and PATCH with a concrete Task: PUT represents a full replacement, while PATCH changes only supplied fields.

WHAT THE INSTRUCTOR DEMONSTRATES

- Inspect a Task instance, serializer data, and rendered JSON
- Implement ModelSerializer fields without exposing editable owner
- Build and route a ModelViewSet
- Trace POST data through validation, save, and response
- Exercise every endpoint in the browsable API or approved API client

PRACTICAL WORK (70%+)

GUIDED START

Participants implement serializer and viewset code in short instructor-led increments. They first make GET list work, then POST create, checking the URL, JSON body, database row, and response status at each step.

PARTICIPANTS THEN BUILD AND VERIFY

- Create TaskSerializer with the approved fields and keep owner and generated fields read-only.
- Build TaskViewSet, register it with a DRF router, and confirm the generated list and detail URLs.
- Use local session login to create and list tasks without allowing the client to choose an owner.
- Implement and exercise retrieve, full update, partial update, and delete behavior.
- Test valid and missing ids, compare returned status codes, and inspect the database after each write.
- Complete the six-operation CRUD checklist and push the Day 2 checkpoint.

PROJECT FEATURE ADDED

Task serializer, router, and endpoints for GET/POST `/api/tasks/` plus GET/PUT/PATCH/DELETE `/api/tasks/{id}/`. Local session login is used only as a development bridge; token security is completed on Day 3.

EXPECTED RESULT

The local API performs every required CRUD operation, returns consistent JSON, assigns an owner server-side, and does not accept owner as an editable request field.

COMPLETION CRITERIA

- List and detail routes resolve without manual URL duplication
- Create returns 201 and the saved record appears in the database
- Retrieve returns 200; unknown id returns 404
- PUT/PATCH changes the intended fields; DELETE returns 204
- Owner is read-only and assigned by server logic
- All six CRUD checks are recorded and the checkpoint is pushed

NEXT-DAY CONNECTION

The CRUD surface now exists, but it is not yet production-safe. Day 3 adds token authentication, per-user query filtering, object permissions, and realistic validation.

DAY 3

AUTHENTICATION, OWNERSHIP, AND VALIDATION

Tuesday, Oct 13

DAILY LEARNING OUTCOME

By the end of the session, participants can secure the API with tokens, isolate each user's data, enforce ownership at query and object level, and return clear validation errors for realistic invalid input.

TEACHING AND PRACTICAL PLAN

TEACHING CONTENT (~30%)

WHAT THE INSTRUCTOR EXPLAINS

- Separate authentication from authorization: authentication answers who the caller is; authorization decides what that identified caller may do.
- Explain a DRF token as a credential sent in the Authorization header on every protected request. Stress that it must be protected like a password.
- Use two users to show the security problem: authentication alone is insufficient if the queryset still returns every user's records.
- Explain defense in depth: filter the queryset to the current user, assign owner on the server, keep owner read-only, and enforce object access consistently.
- Explain validation as part of the API contract. Clients should receive a 400 response with useful field-level errors rather than saving impossible or ambiguous data.
- Clarify 401, 403, and 404 using the program's agreed behavior so participants can diagnose security failures correctly.

WHAT THE INSTRUCTOR DEMONSTRATES

- Configure `rest_framework.authentication` and token endpoint
- Authenticate two users and compare their task lists
- Demonstrate the insecure base queryset, then filter by `request.user`
- Make owner read-only and assign it in `perform_create`
- Reject blank title, invalid status, and past due date with field errors

PRACTICAL WORK (70%+)

GUIDED START

Participants configure token authentication together, create tokens for two supplied accounts, and send authenticated requests. The instructor pauses after each security layer for a deliberate attack-and-fix check.

PARTICIPANTS THEN BUILD AND VERIFY

- Enable DRF token support, migrate its tables, add the token endpoint, and obtain tokens for two test users.
- Send protected requests with and without the Authorization header and compare the responses.
- Filter Task querysets by `request.user` and assign owner automatically during creation.
- Attempt to retrieve, update, and delete another user's task; adjust permissions/query behavior until every attempt is blocked.
- Add validation for nonblank title, allowed status, and creation due date; inspect the returned error JSON.
- Run the complete two-user security scenario and push the Day 3 checkpoint.

PROJECT FEATURE ADDED

Token login endpoint; protected task routes; automatic owner assignment; per-user list and detail isolation; cross-user update/delete protection; validation for nonblank title, approved status values, and due date on creation.

EXPECTED RESULT

Anonymous requests are rejected. User A sees and changes only User A's tasks. User B cannot retrieve or modify User A's records. Invalid payloads return 400 with clear field-level JSON errors.

COMPLETION CRITERIA

- Valid credentials return a token; protected routes reject no token with 401
- Authenticated creation ignores any client-supplied owner value
- Each user's list contains only that user's records
- Cross-user retrieve/update/delete is blocked without leaking data
- Blank title, invalid status, and past creation due date return 400
- Two-user verification evidence is saved and checkpoint is pushed

NEXT-DAY CONNECTION

Day 4 converts today's manual security and validation checks into repeatable automated tests, then packages the project for another reviewer and the deployment environment.

DAY 4

TESTING, QUALITY, DOCUMENTATION, AND DELIVERY

Wednesday, Oct 14

DAILY LEARNING OUTCOME

By the end of the session, participants can verify API behavior with automated tests, document a reproducible setup, protect configuration secrets, and complete a deployment rehearsal with production-ready settings.

TEACHING AND PRACTICAL PLAN

TEACHING CONTENT (~30%)

WHAT THE INSTRUCTOR EXPLAINS

- Explain a test as an executable promise: arrange known data, perform one action, and assert the exact response and database effect.
- Distinguish positive tests from failure tests. A secure API must prove both that allowed work succeeds and forbidden or invalid work fails correctly.
- Explain test isolation: each test creates its own users and tasks so results do not depend on execution order or a developer database.
- Connect manual Day 3 checks to automation: authentication, owner assignment, own-only lists, cross-user denial, CRUD, and validation become repeatable cases.
- Explain production configuration: DEBUG is disabled, secrets and database values come from environment variables, and the deployment installs recorded dependencies.
- Frame the README as part of the product: another developer should be able to clone, configure, test, and use the API without verbal rescue.

WHAT THE INSTRUCTOR DEMONSTRATES

- Create reusable test setup with two users and owned tasks
- Write a 401 test and a cross-user access test
- Read a failing assertion and fix the application, not the test expectation
- Move sensitive settings to environment variables
- Run setup from README steps and walk through deployment configuration

PRACTICAL WORK (70%+)

GUIDED START

Participants build the first three tests with the instructor, predicting each response before running it. Pairs then exchange repositories and follow only the README to reveal missing setup steps.

PARTICIPANTS THEN BUILD AND VERIFY

- Create isolated test users, tokens, and Task fixtures in DRF APITestCase setup.
- Write the first authentication, create/owner, and cross-user isolation tests with the instructor.
- Complete at least eight tests covering the required CRUD, security, and validation behaviors.
- Run the suite repeatedly, read each failure, and correct application behavior instead of weakening the expected result.
- Complete README instructions, dependencies, `.env.example`, secret exclusions, and production settings.
- Exchange repositories for a fresh-setup review and rehearse the approved cloud deployment steps.

PROJECT FEATURE ADDED

At least eight automated API tests; complete README; dependency file; `.env.example`; ignored real environment file; production-safe settings; approved-platform configuration; rehearsed release commands.

EXPECTED RESULT

The full test suite passes repeatedly, a peer can configure the project from the README, secrets remain outside Git, and the project can enter Day 5 without unresolved deployment prerequisites.

COMPLETION CRITERIA

- Tests cover unauthenticated access, create/owner assignment, own-only list, cross-user access, update, delete, and validation
- At least eight meaningful tests pass with none of the required tests skipped
- Fresh setup follows documented commands without instructor correction
- README documents auth and every required endpoint with examples
- Production configuration reads secrets and database settings from environment
- Cloud account, service, database, and release commands are verified

NEXT-DAY CONNECTION

Day 5 uses the tested release package and rehearsed configuration to deploy quickly, leaving enough time for production smoke tests, fixes, final quality improvements, and demonstration.

DAY 5

DEPLOY, VERIFY, IMPROVE, AND DEMONSTRATE

Thursday, Oct 15

DAILY LEARNING OUTCOME

By the end of the session, participants can release the API to a public HTTPS environment, verify production behavior and persistence, correct deployment issues, and demonstrate the completed system with evidence.

TEACHING AND PRACTICAL PLAN

TEACHING CONTENT (~30%)

WHAT THE INSTRUCTOR EXPLAINS

- Explain deployment as the same application running under production configuration: code, dependencies, environment variables, database, migrations, and start command must all agree.
- Describe the release sequence before touching the platform: connect the repository, provision the service/database, set variables, build, migrate, start, then verify.
- Teach log reading as evidence-based troubleshooting: identify whether a failure belongs to configuration, dependency installation, application startup, host settings, or database connectivity.
- Explain a smoke test as a short production confidence check that covers authentication, one full CRUD path, validation, data isolation, and persistence.
- Show how to demonstrate technically: state the claim, perform the action on the live system, and point to visible proof rather than explaining only from slides.
- Reinforce scope discipline: required evidence comes first; optional polish begins only after every completion gate passes.

WHAT THE INSTRUCTOR DEMONSTRATES

- Connect repository, service, database, and environment variables
- Run deployment and production migrations; create/verify a test user
- Use logs to fix one missing-variable or host configuration error
- Run the required live flow against the public URL
- Show passing tests, data isolation, persistence, and repository evidence

PRACTICAL WORK (70%+)

GUIDED START

Participants deploy in small support groups using the release runbook. At each gate they record the URL or output as evidence. Instructor troubleshooting prioritizes shared blockers, then individual application defects.

PARTICIPANTS THEN BUILD AND VERIFY

- Connect the repository to the approved platform, provision the database, and configure every required environment variable.
- Deploy, run production migrations, and obtain a working public HTTPS URL.
- Use platform logs and the troubleshooting matrix to resolve build, startup, host, or database failures.
- Run the production smoke-test sheet for token auth, CRUD, validation, cross-user isolation, and persistence after restart/redeploy.
- Finish README and evidence; make one small quality improvement only after the required checklist passes.
- Demonstrate the live flow at a review station and submit repository URL, deployment URL, passing tests, and completion checklist.

PROJECT FEATURE ADDED

Public HTTPS API with persistent database; completed production configuration; successful smoke-test record; final README and evidence; any approved small polish improvement; completed five-minute demonstration flow.

EXPECTED RESULT

The public service issues tokens, performs owned CRUD, rejects invalid or unauthorized requests, retains data, and can be reproduced from the repository. The participant submits all evidence and completes the live demonstration.

COMPLETION CRITERIA

- Public HTTPS URL is reachable and production migrations are current
- Token authentication and all required CRUD operations work in production
- Two-user isolation and validation checks pass against the live URL
- A created record remains after restart/redeploy verification
- Automated tests pass and README contains final repository and deployment instructions
- Submission includes repository URL, deployment URL, test evidence, and completed checklist
- Live demo completes without instructor code changes

NEXT-DAY CONNECTION

Program completion: the participant leaves with a portfolio-ready backend project and a repeatable workflow for designing, securing, testing, documenting, and deploying future REST APIs.